

Лабораторна робота №8

Тема: Базова концепція фреймворку Electron

1. Перед початком роботи необхідно встановити Node.js (<https://nodejs.org/uk/>) та git (<https://git-scm.com/>)
2. Створіть на комп'ютері каталог (наприклад, папка Electron)
3. Встановіть Electron, використовуючи термінал у Вашому редакторі коду (Atom, VS Code, WEB-Storm і т.д.) *npm i -g electron*

```
PS C:\Electron> npm i -g electron  
  
changed 23 packages in 3s  
  
2 packages are looking for funding  
run `npm fund` for details
```

4. Встановіть ElectronForge

npm i -g create-electron-app

```
npm warn deprecated boolean@3.2.0: Pac  
npm warn deprecated tar@6.2.1: Old ver  
t version. Please update. Support for  
  
changed 464 packages in 23s  
  
87 packages are looking for funding  
run `npm fund` for details
```

5. Ініціалізуємо новий проект

create-electron-app my-app

```
PS C:\Electron> create-electron-app my-app  
  
✓ Resolving package manager: npm v11.12.1  
✓ Resolving template: base  
  > Using @electron-forge/template-base (local module)  
✓ Resolving Electron version: latest  
  > Using Electron version tag: latest  
✓ Initializing directory  
✓ Initializing git repository  
✓ Preparing template  
✓ Initializing template  
✓ Installing template dependencies  
  
PS C:\Electron> cd my-app\  
  
PS C:\Electron\my-app> npm run start
```

Тепер у вас має бути каталог `my-app` з мінімальним шаблоном програми Electron усередині. Якщо ви увійдете в цей каталог і запустите програму, у вас буде все готове для початку розробки.

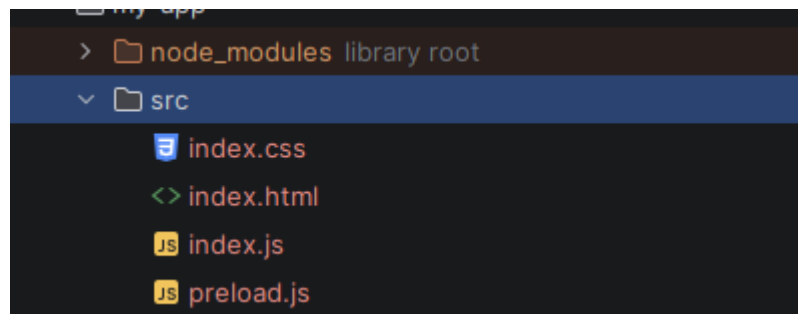
Ініціалізований додаток на даний момент складається з декількох основних файлів:

Index.html

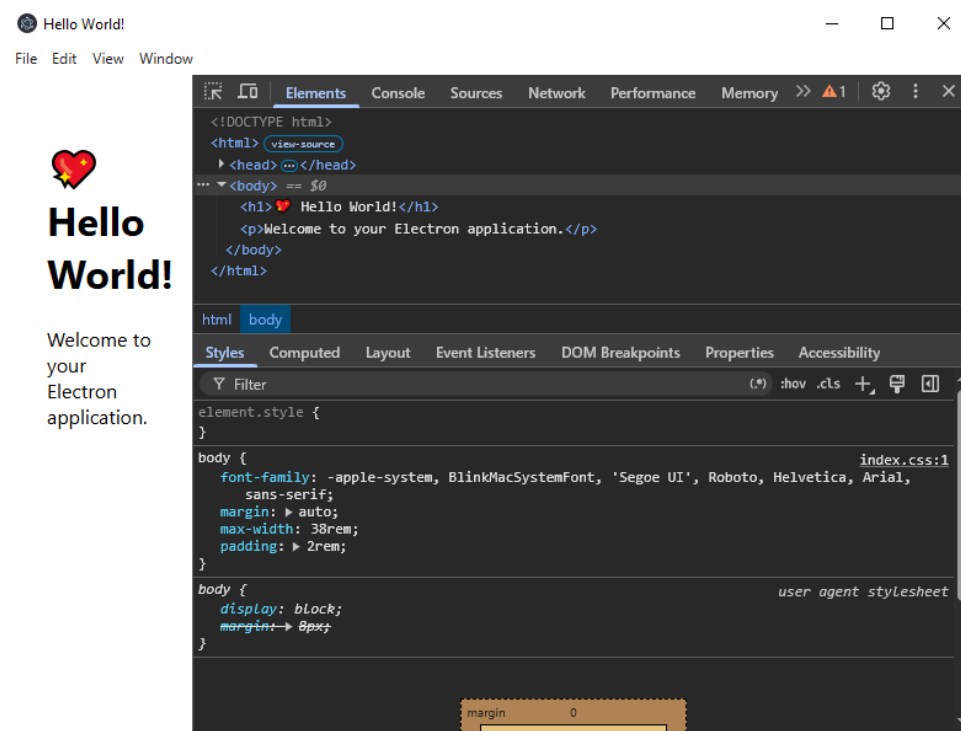
Index.js

Preload.js

Index.css



6. Запуск програми `cd my-app npm run start`



Лабораторна робота №9

Тема: Розробка Electron додатку контролю за CPU

7. Встановити бібліотеку Nodejs під назвою os-utils

```
PS C:\Electron\my-app> npm install os-utils

added 1 package, and audited 540 packages in 4s

89 packages are looking for funding
  run `npm fund` for details

29 vulnerabilities (6 low, 23 high)
```

8. У файл index.js імпортуємо цю бібліотеку до змінної os

```
const os = require('os-utils');
```

9. У цьому файлі реалізуємо функцію зчитування параметрів системи: процесора, пам'яті і дізнаємося обсяг оперативної пам'яті на комп'ютері.

```
const createWindow= ()=>{
  // Create the browser window.
  const mainWindow=new BrowserWindow({
    width: 1000,
    height: 400,
    icon: __dirname+"/images.jpg",
    webPreferences:{
      nodeIntegration: true
    }
  });
  // and load the index.html of the app.
  mainWindow.loadFile(path.join(__dirname,"index.html"));
  os.cpuUsage(function(v) {
    console.log("CPU Usage (%): " + v * 100);
    console.log("Mem Usage (%): " + os.freememPercentage() * 100);
    console.log("Total Mem (GB): " + os.totalmem() / 1024);
  });
}
```

Запустимо програму знову

npm run start

Поки інформація про параметри буде видно в консолі, а не в самому додатку. Це тому, що файли index.js і index.html запущені в різних процесах (Mainprocess і Renderer процеси).

```
CPU Usage (%): 32.09846153846154  
Mem Usage (%): 40.68601204360964  
Total Mem (GB): 15.944709777832031
```

!Самостійно. Вивести інформацію в консоль про поточну систему, наприклад:

```
✓ Launched Electron app. Type rs in  
  
CPU Usage (%): 27.78186274509804  
Mem Usage (%): 39.27566085539773  
Total Mem (GB): 15.944709777832031  
Platform: win32
```

10. Для взаємодії між процесами рендерингу із головного процесу необхідно використовувати модуль ipcMain.

Для надсилання даних із головного процесу в браузер потрібно скористатися методом send()

```
os.cpuUsage(function(v) {  
  console.log("CPU Usage (%): "+v*100);  
  mainWindow.webContents.send("cpu", v * 100);  
  console.log("Mem Usage (%): "+os.freememProcentage() *100);  
  mainWindow.webContents.send("mem", os.freememPercentage() * 100);  
  console.log("Total Mem (GB): "+os.totalmem() /1024);  
  mainWindow.webContents.send("total-mem", os.totalmem() / 1024);  
});
```

Далі необхідно доповнити властивостями об'єкт BrowserWindow для включення інтеграції процесів (основного та графічного) після чого ми зможемо використовувати nodejs модуль у html файлі для отримання повідомлень. Для цього задають властивість webPreferences у BrowserWindow:

```
const createWindow = () => {
  // Create the browser window.
  const mainWindow = new BrowserWindow({
    width: 800,
    height: 600,
    webPreferences: {
      preload: path.join(__dirname, "preload.js"),
      nodeIntegration: true,
    },
  });
};
```

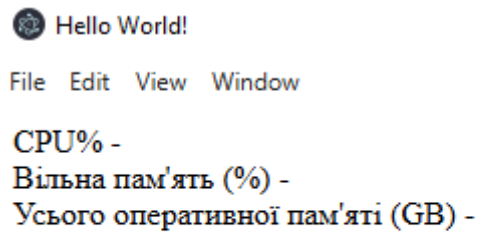
11. Відкриємо файл preload.js та імпортуємо electron. Модуль ipcRenderer є EventEmitter. Він включає ряд методів, для відправки синхронних і асинхронних повідомлень з графічного (веб-сторінки) в основний процес. Також можна отримувати відповіді (слухати) від головного процесу.

```
const electron = require("electron");
const ipcRenderer = electron.ipcRenderer;
ipcRenderer.on("cpu", (event, data) => {
  console.log("cpu", data);
});
ipcRenderer.on("mem", (event, data) => {
  console.log("mem", data);
});
ipcRenderer.on("total-mem", (event, data) => {
  console.log("total-mem", data);
});
```

12. Створимо інтерфейс користувача для виведення інформації про параметри CPU у вікні програми. Для цього внесемо наступний HTML код у файл index.html замість HelloWorld.

```
<div class="box">
  <span class="label">CPU%</span>
  <span id="cpu">-</span>
</div>
<div class="box">
  <span class="label">Вільна пам'ять (%)</span>
  <span id="mem">-</span>
</div>
<div class="box">
  <span class="label">Усього оперативної пам'яті (GB)</span>
  <span id="total-mem">-</span>
</div>
```

В результаті додаток матиме вигляд:



13. Відредагуємо стилі для програми у файлі index.css

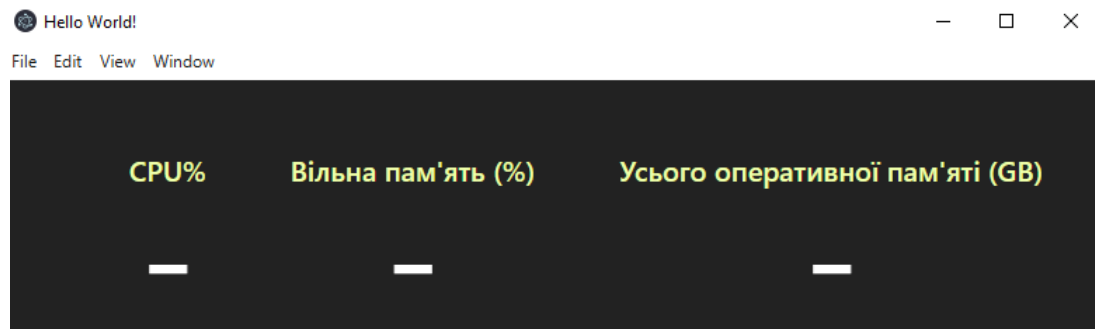
```
html{
  height:100vh;
}

body{
  font-family: -apple-system, BlinkMacSystemFont,"Segoe UI", Roboto,Helvetica,
  Arial,sans-serif;
  margin:auto;
  width:100%;
  height:100%;
  padding:3%;
  overflow:hidden;
  background-color:#222;
  display:flex;
  justify-content:center;
  align-items:center;
  color:white;
}

.box{
  display:flex;
  justify-content:center;
  align-items:center;
  color:white;
  flex-direction:column;
  font-size:100px;
  line-height:0.95;
  padding:030px;
}

.label{
  font-size:20px;
  font-weight:600;
  color:#efa;
}
```

14. У результаті додаток набуде вигляду:



15. Дані поки виводяться в консоль, тому наступним кроком необхідно зробити перенесення даних з основного процесу підготовлений вже елемент `span` з відповідним `id`. Для цього використовується метод `getElementById` для вибору з дерева DOM необхідного елемента і методу вбудованого HTML для вставки даних у відповідний вузол.

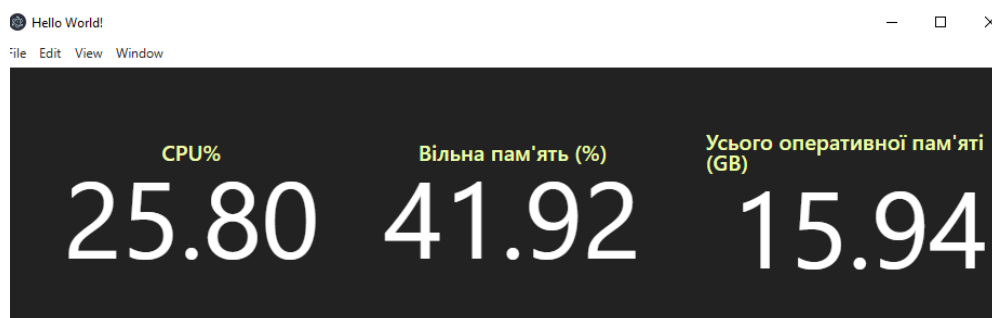
```
const electron = require("electron");
const ipcRenderer = electron.ipcRenderer;

ipcRenderer.on("cpu", (event: IpcRendererEvent, data: void) => {
  document.getElementById("cpu").innerHTML = data.toFixed(2);
  console.log("cpu", data);
});

ipcRenderer.on("mem", (event: IpcRendererEvent, data: void) => {
  document.getElementById("mem").innerHTML = data.toFixed(2);
  console.log("mem", data);
});

ipcRenderer.on("total-mem", (event: IpcRendererEvent, data: void) => {
  document.getElementById("total-mem").innerHTML = data.toFixed(2);
  console.log("total-mem", data);
});
```

Результат:

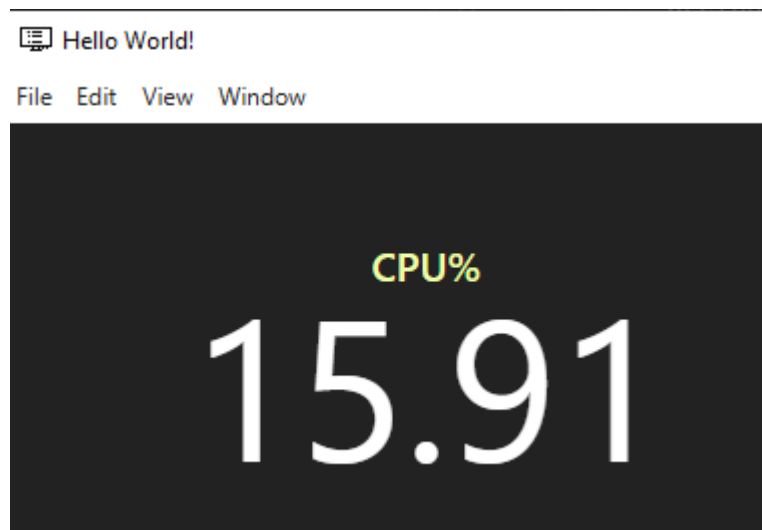


!Самостійно. Вивести дані із округленням до 2 знаків, як на рис. вище.

16. Зараз при запуску програми інформація про стан CPU статична. Тому модифікуємо код index.js для оновлення інформації через кожну секунду за допомогою методу setInterval().

```
setInterval(() => {  
  os.cpuUsage(function (v) {  
    mainWindow.webContents.send("cpu", v * 100);  
    mainWindow.webContents.send("mem", os.freememPercentage() * 100);  
    mainWindow.webContents.send("total-mem", os.totalmem() / 1024);  
  });  
}, 1000);
```

17. Поміняємо іконку програми на свою (знайдіть будь-яку в інтернеті)



18. За замовчуванням у програмі під час запуску відображається інструмент розробника, щоб приховати його потрібно закоментувати наступний запис

```
// mainWindow.webContents.openDevTools();
```

Або додати наступний код:

```
mainWindow.webContents.on("devtools-opened", ()=>{  
  mainWindow.webContents.closeDevTools();  
});
```

19. Далі необхідно запакувати додаток і створити інсталятор

npm run make

Після запуску цієї команди буде створено каталог out в якому буде два каталоги в одному з них інсталятор для поточної системи, а в іншому вже встановлений додаток

libEGL.dll	21.05.2026 23:35	Расширение при...	496 КБ
libGLESv2.dll	21.05.2026 23:35	Расширение при...	7 917 КБ
LICENSE	21.05.2026 23:35	Файл	2 КБ
LICENSES.chromium.html	21.05.2026 23:35	Firefox HTML Doc...	19 890 КБ
my-app.exe	21.05.2026 23:35	Приложение	221 200 КБ
resources.pak	21.05.2026 23:35	Файл "PAK"	6 713 КБ
snapshot_blob.bin	21.05.2026 23:35	Файл "BIN"	339 КБ
v8_context_snapshot.bin	21.05.2026 23:35	Файл "BIN"	705 КБ
version	21.05.2026 23:35	Файл	1 КБ

Скористаємося іншим пакувальником:

npm i electron-builder --dev

У файлі package.json змінимо ім'я програми на CPU Monitor, за бажанням змініте опис програми у значенні ключа "description"

```
{
  "name": "my-app",
  "productName": "CPU Monitor",
  "version": "1.0.0",
  "description": "My Electron application description",
  "main": "src/index.js",
  "private": true,
  "scripts": {
```

20. Пропишемо нове значення у полі "scripts": файлу package.json для нового пакувальника. Для цього вставте наступний код:

```
    "scripts": {
      "start": "electron-forge start",
      "package": "electron-forge package",
      "make": "electron-forge make",
      "publish": "electron-forge publish",
      "lint": "echo\\\"No linting configured\\\"",
      "build-installer": "electron-builder"
    },
```

21. Далі задамо налаштування білдера в цьому ж файлі для створення повноцінного інсталятора (опис значення властивостей в наступній презентації до лекції)

```
"build": {
  "appId": "my-app",
  "win": {
    "target": [
      "nsis"
    ],
    "requestedExecutionLevel": "requireAdministrator"
  },
  "nsis": {
    "uninstallDisplayName": "CPU Monitor",
    "license": "license.txt",
    "oneClick": false,
    "allowToChangeInstallationDirectory": true
  }
},
```

22. Створіть файл license.txt у папці my-app і вставте, наприклад, цей текст:

"Коробковий" продукт застосовується в основному для роздрібної торгівлі та зручний для приватних осіб або малого бізнесу. Дозвіл на використання програмного продукту на одному комп'ютері дає покупка однієї «коробки» і не важливо, скільки людей користуватиметься цим ПК. Також можна змінити ПК, але кілька разів

23. Запустимо пакувальник

npm run build-installer

В результаті буде створено папку dist з повноцінним інсталятором програми.

Имя	Дата изменения	Тип	Размер
win-unpacked	22.05.2026 00:10	Папка с файлами	
builder-debug.yml	22.05.2026 00:10	Файл "YML"	8 КБ
builder-effective-config.yaml	22.05.2026 00:10	Файл "YAML"	1 КБ
CPU Monitor Setup 1.0.0.exe	22.05.2026 00:10	Приложение	98 464 КБ
CPU Monitor Setup 1.0.0.exe.blockmap	22.05.2026 00:10	Файл "BLOCKMAP"	103 КБ

!Самостійно. З поточними налаштуваннями інсталятор автоматично створює ярлик на робочому столі та в меню Пуск. Зробити таким чином, щоб користувач при установці сам міг задавати ці параметри.

